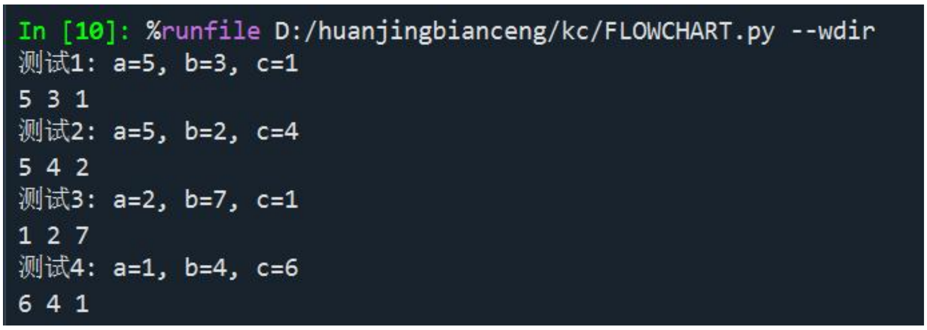
1. 流程图逻辑

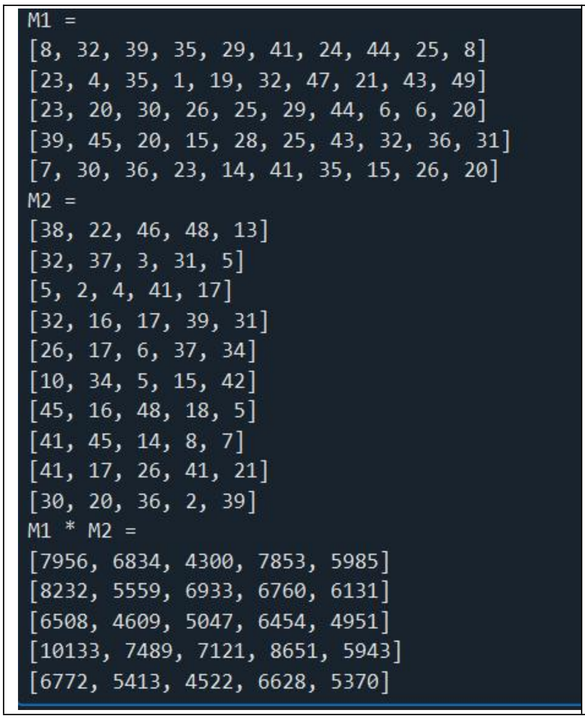
函数严格按照题目给出的流程图编写，利用嵌套 if 条件语句对三个输入数字 a、b、c 进行大小比较，并将数字按降序打印输出。程序首先判断 a 是否大于 b，若 a>b 成立，则进一步判断 b 与 c 的大小关系：当 b>c 时，打印 a,b,c；若 b>c 不成立，则继续比较 a 和 c，当 a>c 为真时打印 a,c,b，a>c 为假时打印 c,a,b。如果初始判断 a>b 不成立，程序进入左侧分支，判断 b 是否大于 c：b>c 成立则打印 b,a,c，不成立则打印 c,b,a。流程图中的紫色平行四边形代表打印操作，按照判定好的顺序输出三个数值。选取多组随机输入进行测试，验证该函数可以正确将三个数字从大到小排序并输出，结果如下。



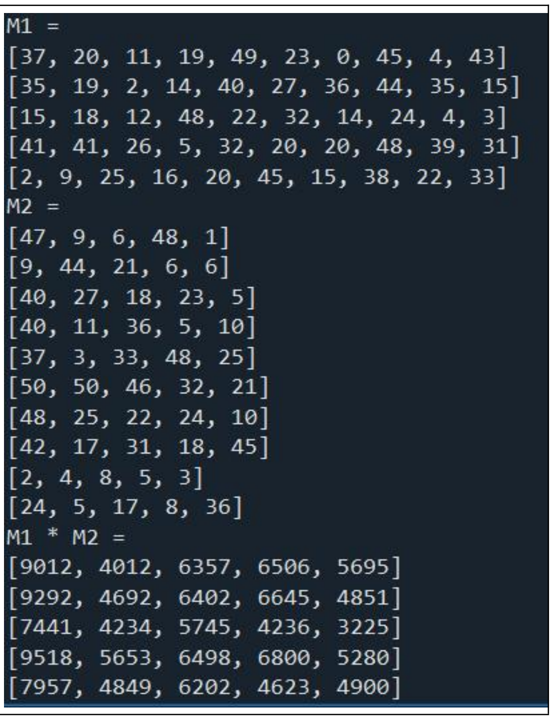
```
In [10]: %runfile D:/huanjingbianceng/kc/FLOWCHART.py --wdir
测试1： a=5， b=3， c=1
5 3 1
测试2： a=5， b=2， c=4
5 4 2
测试3： a=2， b=7， c=1
1 2 7
测试4： a=1， b=4， c=6
6 4 1
```

2.矩阵运算

本部分编写了Matrixmultip(M1,M2)函数来完成矩阵乘法，编写过程中只使用for循环、加法和乘法，不直接调用现成的矩阵运算函数。首先生成两个随机整数矩阵M1和M2，M1是5行10列，M2是10行5列，矩阵里面的元素都是0到50之间的随机整数。根据矩阵乘法规则，结果矩阵中第i行第 列的值，是M1第i行和M2第j列对应位置数字相乘之后相加得到的。我使用多层for循环遍历矩阵的行与列，逐个算出结果矩阵里每一个元素，最后返回相乘得到的新矩阵。构建五行十列和十行五列矩阵并且进行运算，矩阵内的数字随机，得出的结果为五行五列矩阵，随机运行两次的结果如下：



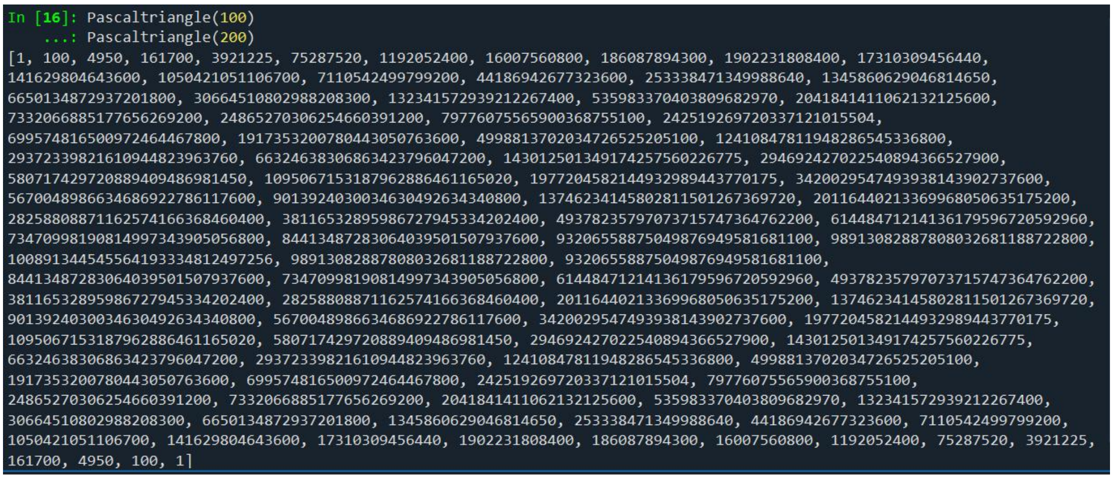
```
M1 =
[8, 32, 39, 35, 29, 41, 24, 44, 25, 8]
[23, 4, 35, 1, 19, 32, 47, 21, 43, 49]
[23, 20, 30, 26, 25, 29, 44, 6, 6, 20]
[39, 45, 20, 15, 28, 25, 43, 32, 36, 31]
[7, 30, 36, 23, 14, 41, 35, 15, 26, 20]
M2 =
[38, 22, 46, 48, 13]
[32, 37, 3, 31, 5]
[5, 2, 4, 41, 17]
[32, 16, 17, 39, 31]
[26, 17, 6, 37, 34]
[10, 34, 5, 15, 42]
[45, 16, 48, 18, 5]
[41, 45, 14, 8, 7]
[41, 17, 26, 41, 21]
[30, 20, 36, 2, 39]
M1 * M2 =
[7956, 6834, 4300, 7853, 5985]
[8232, 5559, 6933, 6760, 6131]
[6508, 4609, 5047, 6454, 4951]
[10133, 7489, 7121, 8651, 5943]
[6772, 5413, 4522, 6628, 5370]
```



```
M1 =
[37, 20, 11, 19, 49, 23, 0, 45, 4, 43]
[35, 19, 2, 14, 40, 27, 36, 44, 35, 15]
[15, 18, 12, 48, 22, 32, 14, 24, 4, 3]
[41, 41, 26, 5, 32, 20, 20, 48, 39, 31]
[2, 9, 25, 16, 20, 45, 15, 38, 22, 33]
M2 =
[47, 9, 6, 48, 1]
[9, 44, 21, 6, 6]
[40, 27, 18, 23, 5]
[40, 11, 36, 5, 10]
[37, 3, 33, 48, 25]
[50, 50, 46, 32, 21]
[48, 25, 22, 24, 10]
[42, 17, 31, 18, 45]
[2, 4, 8, 5, 3]
[24, 5, 17, 8, 36]
M1 * M2 =
[9012, 4012, 6357, 6506, 5695]
[9292, 4692, 6402, 6645, 4851]
[7441, 4234, 5745, 4236, 3225]
[9518, 5653, 6498, 6800, 5280]
[7957, 4849, 6202, 4623, 4900]
```

3.帕斯卡三角

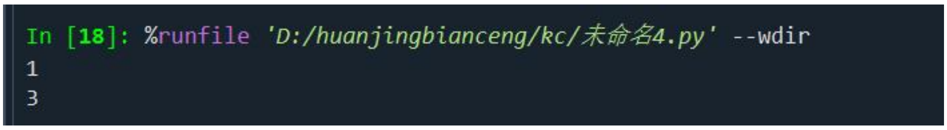
通过编写 Pascaltriangle(k)函数用来输出杨辉三角的第 k 行。杨辉三角的规律是每行首尾数字都是 1，中间每个数字等于上一行相邻两个数字之和。在代码里利用这个递推规则不断生成新一行的数据，最后打印出目标的第 k 行。我分别调用 Pascaltriangle(100)和 Pascaltriangle(200)进行测试，输出对应的结果，以此验证函数能够正常生成杨辉三角指定行数。由于杨辉三角第两百项数据过大，列举第一百项运行结果，如下所示：



```
In [16]: Pascaltriangle(100)
    ...: Pascaltriangle(200)
[1, 100, 4950, 161700, 3921225, 75287520, 1192052400, 16007560800, 186087894300, 1902231808400, 17310309456440,
141629804643600, 1050421051106700, 7110542499799200, 44186942677323600, 253338471349988640, 1345860629046814650,
6650134872937201800, 30664510802988208300, 132341572939212267400, 535983370403809682970, 2041841411062132125600,
7332066885177656269200, 24865270306254660391200, 79776075565900368755100, 242519269720337121015504,
699574816500972464467800, 1917353200780443050763600, 4998813702034726525205100, 12410847811948286545336800,
29372339821610944823963760, 66324638306863423796047200, 143012501349174257560226775, 294692427022540894366527900,
580717429720889409486981450, 1095067153187962886461165020, 1977204582144932989443770175, 3420029547493938143902737600,
5670048986634686922786117600, 9013924030034630492634340800, 13746234145802811501267369720, 20116440213369968050635175200,
28258808871162574166368460400, 38116532895986727945334202400, 49378235797073715747364762200, 61448471214136179596720592960,
73470998190814997343905056800, 84413487283064039501507937600, 93206558875049876949581681100, 98913082887808032681188722800,
100891344545564193334812497256, 98913082887808032681188722800, 93206558875049876949581681100,
84413487283064039501507937600, 73470998190814997343905056800, 61448471214136179596720592960, 49378235797073715747364762200,
38116532895986727945334202400, 28258808871162574166368460400, 20116440213369968050635175200, 13746234145802811501267369720,
9013924030034630492634340800, 5670048986634686922786117600, 3420029547493938143902737600, 1977204582144932989443770175,
1095067153187962886461165020, 580717429720889409486981450, 294692427022540894366527900, 143012501349174257560226775,
66324638306863423796047200, 29372339821610944823963760, 12410847811948286545336800, 4998813702034726525205100,
1917353200780443050763600, 699574816500972464467800, 242519269720337121015504, 79776075565900368755100,
24865270306254660391200, 7332066885177656269200, 2041841411062132125600, 535983370403809682970, 132341572939212267400,
30664510802988208300, 6650134872937201800, 1345860629046814650, 253338471349988640, 44186942677323600, 7110542499799200,
1050421051106700, 141629804643600, 17310309456440, 1902231808400, 186087894300, 16007560800, 1192052400, 75287520, 3921225,
161700, 4950, 100, 1]
```

4.加法或加倍

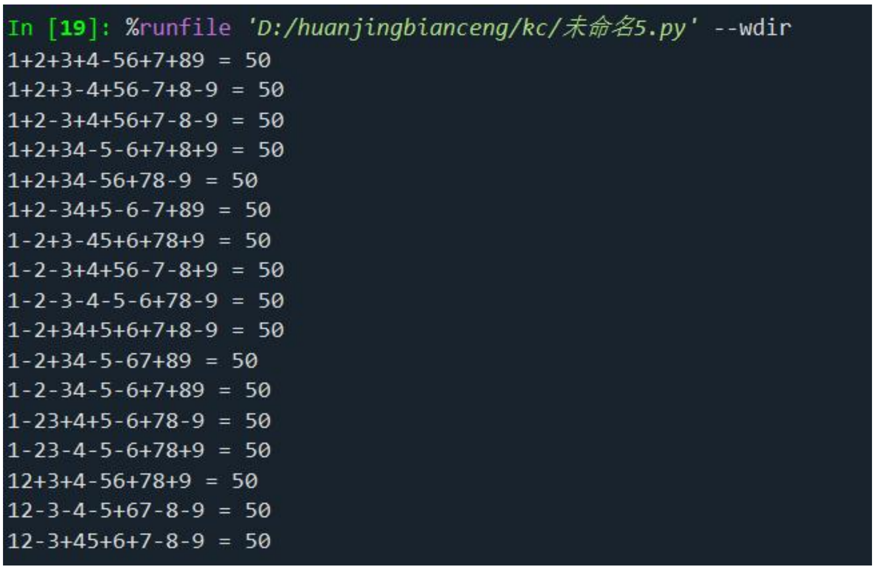
用来求解从初始 1 元开始,通过每次选择将钱数翻倍或者加 1 元，凑到恰好 x 元所需要的最少操作步数。解题思路是从目标数字反向推导，如果当前数是偶数就除以 2，若是奇数就减 1，每进行一次操作就计数加一，直到数字回到 1，统计出总步数。我选取随机整数 x 进行测试，例如输入 Least_moves(2)得到结果 1，Least_moves(5)得到结果 3，以此检验函数能否正确求出最少移动次数，运行结果如下。



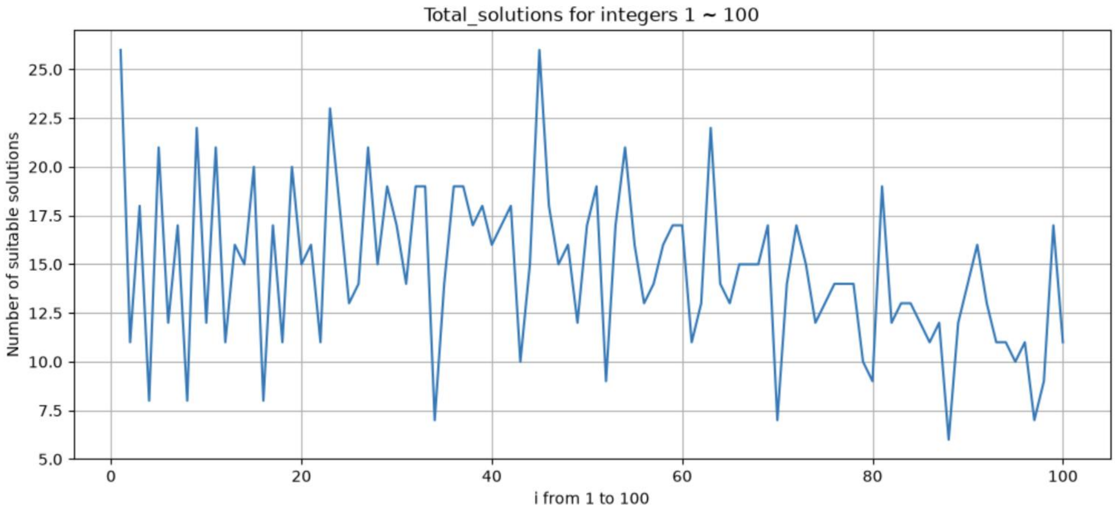
```
In [18]: %runfile 'D:/huanjingbianceng/kc/未命名4.py' --wdir
1
3
```

5.动态规划

在数字串 123456789 的数字间隙中添加加号、减号或者直接拼接相邻数字，保持数字原有顺序，找出所有计算结果等于目标值的表达式并打印出来。接着统计 1 到 100 每个整数对应的合法表达式数量，将统计结果存入列表 Total_solutions，再使用绘图工具画出该列表的变化曲线，最后找出解数量最多和最少对应的目标数字。我以目标值 50 作为例子运行代码，成功输出符合要求的表达式，完成全部测试，结果如下。



```
In [19]: %runfile 'D:/huanjingbianceng/kc/未命名5.py' --wdir
1+2+3+4-56+7+89 = 50
1+2+3-4+56-7+8-9 = 50
1+2-3+4+56+7-8-9 = 50
1+2+34-5-6+7+8+9 = 50
1+2+34-56+78-9 = 50
1+2-34+5-6-7+89 = 50
1-2+3-45+6+78+9 = 50
1-2-3+4+56-7-8+9 = 50
1-2-3-4-5-6+78-9 = 50
1-2+34+5+6+7+8-9 = 50
1-2+34-5-67+89 = 50
1-2-34-5-6+7+89 = 50
1-23+4+5-6+78-9 = 50
1-23-4-5-6+78+9 = 50
12+3+4-56+78+9 = 50
12-3-4-5+67-8-9 = 50
12-3+45+6+7-8-9 = 50
```



```
Maximum number of solutions = 26, corresponding i: [1, 45]
Minimum number of solutions = 6, corresponding i: [88]
```